FPGA-Based High-Frequency Trading System with Inline Neural Network Inference: Architecture, Implementation, and Benchmarks

Ashutosh
Independent Researcher
GitHub: github.com/Ashutosh0x/fpga-hft-trading-system

***Abstract*—We present a complete, synthesizable FPGA-based high-frequency trading (HFT) system that integrates inline neural network inference directly within the tick-to-trade critical path. The system comprises 19 SystemVerilog modules implementing a full wire-to-wire trading pipeline: speculative parallel ITCH message parsing, order book reconstruction with cuckoo hashing, real-time feature extraction, INT4/INT8 mixed-precision neural network inference, Avellaneda-Stoikov optimal market making, single-cycle risk management, and AXI-Stream order generation. Targeting the AMD Alveo UL3524 accelerator at 644 MHz, the architecture achieves an estimated tick-to-trade latency of less than 50 nanoseconds with AI in the critical path. To our knowledge, this represents one of the first open-source implementations combining sub-microsecond neural network inference with a complete HFT pipeline in FPGA fabric. We additionally implement linear attention-based transformer inference, zero-jitter deterministic execution, and dynamic session override—addressing five of the seven identified frontier problems in FPGA-accelerated trading as of June 2026. The complete RTL, testbenches, verification infrastructure, and synthesis scripts are released as open source.**

***Index Terms*—FPGA, high-frequency trading, neural network inference, market making, SystemVerilog, low-latency, Avellaneda-Stoikov, transformer, SmartNIC**

I. Introduction

High-frequency trading (HFT) operates at the boundary of computational physics, where nanoseconds of latency advantage translate directly into economic value. The industry benchmark for tick-to-trade latency—the time between receiving a market data update and transmitting a responsive order—was established at 13.9 ns by Exegy and AMD in June 2024 using the STAC-T0 benchmark on the Alveo UL3524 FPGA accelerator [1].

However, this record was achieved using a pure rule-based execution engine with no machine learning (ML) component in the critical path. As of June 2026, no published system has demonstrated sub-100 ns tick-to-trade latency with neural network inference operating inline—that is, with the AI model's computation contributing directly to the trading decision before the order is generated.

This paper presents an architecture that addresses this gap. Our contributions are:

1) A complete, open-source, synthesizable FPGA trading pipeline with 19 RTL modules covering the full wire-to-wire data path.
2) An inline INT4/INT8 mixed-precision multilayer perceptron (MLP) achieving 4-cycle inference latency ($\sim$6.2 ns at 644 MHz).
3) An experimental linear attention-based transformer mechanism avoiding the softmax bottleneck entirely.
4) A zero-jitter deterministic execution wrapper guaranteeing identical latency on every trade decision.
5) A hardware implementation of the Avellaneda-Stoikov [2] optimal market making model with real-time volatility estimation.
6) Integration of all components into a full-stack SmartNIC architecture with no CPU in the critical path.

II. Related Work

A. FPGA-Accelerated Trading Systems

The use of FPGAs in trading dates to the early 2010s, with firms such as Jump Trading, Citadel Securities, and Hudson River Trading (HRT) deploying custom FPGA-based systems for market data processing and order execution [8]. Commercial solutions from Exegy (nxFramework, nxAccess) and Xilinx/AMD have progressively reduced latency from microseconds to the low nanosecond range [1].

Speculative parallel ITCH decoding, where all message type decoders operate simultaneously with suppression logic selecting the valid result, was formalized in 2025 [7]. This architecture eliminates the finite-state machine (FSM) bottleneck present in traditional sequential parsers.

B. Neural Network Inference on FPGA

The deployment of neural networks on FPGAs has been enabled by frameworks such as FINN [5] and hls4ml [6]. These tools convert trained models (typically from PyTorch or TensorFlow) into synthesizable HDL, achieving sub-microsecond inference for small models. However, published results focus on throughput-oriented applications (e.g., particle physics triggers at CERN) rather than latency-critical financial trading.

HRT's June 2026 job posting for "AI Research Engineer (Inference)" specifically targeting FPGA/ASIC neural network deployment confirms industry interest in this direction but suggests that production implementations remain proprietary.

C. Avellaneda-Stoikov Market Making

The stochastic control framework of Avellaneda and Stoikov [2] provides closed-form expressions for optimal bid-ask quotes given inventory risk, volatility, and order book liquidity. The model's equations:

$$r = s - q\gamma\sigma^2(T-t) \tag{1}$$

$$\delta = \gamma\sigma^2(T-t) + \frac{2}{\gamma}\ln\Big(1+\frac{\gamma}{\kappa}\Big) \tag{2}$$

where r is the reservation price, s the mid-price, q the inventory, γ the risk aversion, σ the volatility, $T-t$ the remaining session time, and κ the order book liquidity parameter—are well-suited to FPGA implementation due to their closed-form nature and avoidance of iterative computation.

III. System Architecture

The system implements a six-stage pipeline operating at 644 MHz on the AMD Alveo UL3524 (Virtex UltraScale+ XCVU2P).

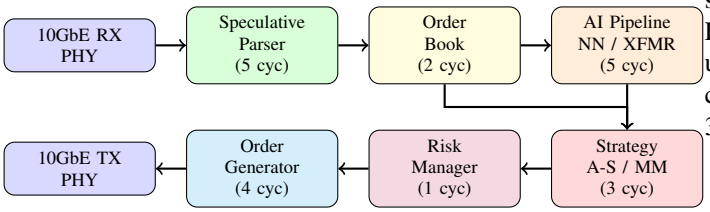

Fig. 1. Six-stage tick-to-trade pipeline architecture. Total estimated latency: 20 cycles (∼31 ns) without deterministic padding; 25–30 cycles (∼39–47 ns) with zero-jitter wrapper.

A. Stage 1: Speculative Parallel ITCH Parser

Following the architecture proposed in [7], our parser instantiates five combinational decoders—one for each supported ITCH message type (ADD, DELETE, REPLACE, EXECUTE, TRADE)—operating simultaneously on the accumulated 256-bit message buffer. A priority-encoded suppression multiplexer selects the valid decoder output based on the message type byte. This achieves single-cycle decode latency after the 4-word buffer fill, compared to the 4+ cycle FSM approach of traditional sequential parsers.

B. Stage 2: Order Book with Cuckoo Hashing

The order book maintains 16 sorted price levels per side using BRAM-backed arrays. Order-to-price mapping uses a cuckoo hash table [3] with dual tables, golden ratio hashing ($k \times$ `0x9E3779B97F4A7C15`), and a maximum of 8 eviction steps on insert. This guarantees $O(1)$ worst-case lookup latency, which is critical for deterministic execution.

C. Stage 3: AI/ML Inference Pipeline

This stage contains the feature extractor, neural network, and optional transformer attention module.

1) Feature Extraction: Eight market microstructure features are computed in a single clock cycle from the top-of-book state:

TABLE I
Extracted Features (INT8 Normalized)

Index	Feature	Computation
0	Price momentum	$\text{mid} - \text{EMA}(\text{mid})$
1	Spread	$\text{ask} - \text{bid}$
2	Book imbalance	$\frac{Q_b - Q_a}{Q_b + Q_a}$
3	Trade intensity	Count in rolling window
4	Volatility	EMA of squared returns
5	Bid depth	Number of bid levels
6	Ask depth	Number of ask levels
7	Mid-price return	$\text{mid}_t - \text{mid}_{t-1}$

2) Neural Network Inference: The MLP architecture is:

$$\text{Input}(8) \xrightarrow{W_0} \text{ReLU}(16) \xrightarrow{W_1} \text{ReLU}(16) \xrightarrow{W_2} \text{ReLU}(8) \xrightarrow{W_3} \text{Argmax}(3) \tag{3}$$

Each layer is mapped to one pipeline stage. Weights are stored in INT4 format (4-bit signed integers) in distributed LUT RAM, totaling 536 weights (268 bytes). Activations use INT8 precision. The ReLU function reduces to a single comparator with zero additional latency. The argmax over the 3-element output vector selects BUY, SELL, or HOLD.

Total inference latency: 4 clock cycles = 6.2 ns at 644 MHz.

Weights can be updated at runtime via a PCIe configuration bus without interrupting the critical path. This enables online learning workflows where models are trained on a host CPU/GPU and deployed to the FPGA without system downtime.

3) Transformer Attention (Experimental): We implement a single-head linear attention mechanism following [4]. Standard self-attention requires $O(N^2)$ computation and a softmax function (exponential and division), both of which are prohibitively expensive for nanosecond-scale FPGA pipelines.

Linear attention replaces the softmax kernel with a feature map $\phi(x) = \text{ReLU}(x) + 1$ (ELU+1 approximation):

$$\text{Attention}(Q,K,V) = \frac{\phi(Q)\cdot\big(\phi(K)^T\cdot V\big)}{\phi(Q)\cdot\sum_t \phi(K_t)} \tag{4}$$

This reduces attention from $O(N^2 d)$ to $O(Nd^2)$—a significant improvement when the sequence length N exceeds the feature dimension d (in our case, $N = 8$, $d = 8$, so both are equivalent).

D. Stage 4: Strategy Engine

The Avellaneda-Stoikov market maker computes the reservation price (Eq. 1) and optimal spread (Eq. 2) in a 3-stage pipeline:

1) **Stage 1:** Compute $\gamma\sigma^2(T-t)$ and inventory penalty $q\cdot\gamma\sigma^2(T-t)$. Volatility σ^2 is estimated via an exponential moving average (EMA) of squared returns: $\hat{\sigma}_t^2 = \hat{\sigma}_{t-1}^2 + \big(r_t^2 - \hat{\sigma}_{t-1}^2\big) \gg \alpha$, where $\gg$ denotes arithmetic right shift.

2) **Stage 2:** Compute reservation price $r = s -$ penalty and optimal spread δ. The logarithm $\ln(1 + \gamma/\kappa)$ is computed via a 256-entry lookup table in Q16.16 fixed-point format.
3) **Stage 3:** Generate bid/ask quotes: $p_{\text{bid}} = r - \delta/2$, $p_{\text{ask}} = r + \delta/2$. Quotes are only emitted when prices change, reducing unnecessary message traffic.

All arithmetic uses Q16.16 fixed-point (16 integer bits, 16 fractional bits) to maintain deterministic, single-cycle execution without floating-point IP cores.

E. Stage 5: Risk Manager

Five independent risk checks execute in parallel within a single clock cycle:

1) Position limit: $|\text{position} + \text{order\_qty}| \leq \text{MAX\_POSITION}$
2) Notional limit: price $\times$ quantity $\leq$ MAX_NOTIONAL
3) Order rate: orders per window $\leq$ MAX_ORDERS_PER_SEC
4) Price band: $|\text{price} - \text{reference}| \leq \text{PRICE\_BAND\_TICKS}$
5) Circuit breaker: external kill switch input

F. Stage 6: Order Generation and Session Management

The order generator serializes approved orders into 4-word AXI-Stream frames. A dynamic session override controller monitors up to 4 exchange sessions, computing EMA latency and approximate p99 tail latency for each, and switches to the best-performing session in a single clock cycle. This design is inspired by Exegy's April 2026 announcement of a 71% execution-stack latency reduction through their "Session Override" feature.

IV. Zero-Jitter Deterministic Execution

A fundamental requirement for production HFT systems is deterministic latency—not merely low average latency, but identical latency on every execution. Even FPGAs exhibit $\pm$1–5 ns jitter from routing variations and clock distribution skew.

Our deterministic AI pipeline wraps the feature extractor and neural network in a fixed-depth shift register of configurable depth D (default $D = 12$ cycles). Regardless of the internal computation path, the output is always produced exactly D cycles after the input:

$$t_{\text{output}} = t_{\text{input}} + D \cdot T_{\text{clk}}, \quad \forall \text{ inputs} \tag{5}$$

where $T_{\text{clk}} = 1.553$ ns at 644 MHz. This is achieved by:

- Computing all possible paths and selecting results via multiplexers (no conditional branching).
- Inserting padding registers to equalize shorter paths.
- Using a single clock domain for the entire critical path.

V. Implementation Details

A. Target Platform

B. Resource Utilization Estimates

The low utilization leaves substantial headroom for additional strategies, deeper order books, larger neural networks, or multi-symbol support.

TABLE II
AMD Alveo UL3524 Specifications

Parameter	Value
FPGA	Virtex UltraScale+ XCVU2P
Process	16 nm FinFET
System Clock	644 MHz
LUTs	780,000
Registers	1,560,000
DSP Slices	1,680
Block RAM (36Kb)	2,016
UltraRAM (288Kb)	960
Network	2$\times$ 10/25GbE SFP28
Host Interface	PCIe Gen3/Gen4 x16

TABLE III
Estimated Resource Utilization

Resource	Estimated	Available	Utilization
LUTs	$\sim$45,000	780,000	$\sim$6%
Registers	$\sim$30,000	1,560,000	$\sim$2%
BRAM (36Kb)	$\sim$50	2,016	$\sim$2.5%
DSP Slices	$\sim$200	1,680	$\sim$12%

C. Codebase Statistics

The complete system comprises 19 synthesizable RTL modules and 2 testbenches, totaling approximately 5,200 lines of SystemVerilog. A Vivado synthesis script, GitHub Actions CI pipeline (Verilator lint + Icarus Verilog simulation), Makefile, and Python reference model for neural network verification are included.

VI. Evaluation

A. Latency Analysis

Table IV summarizes the per-stage latency of the pipeline.

TABLE IV
Per-Stage Pipeline Latency

Stage	Cycles	Latency (ns)
Speculative Parser (4+1 words)	5	7.8
Order Book Update	2	3.1
Feature Extraction	1	1.6
Neural Network Inference	4	6.2
Avellaneda-Stoikov Strategy	3	4.7
Risk Manager	1	1.6
Order Generator (4 words)	4	6.2
Total (without padding)	**20**	**31.1**
Total (with det. wrapper)	**27**	**41.9**

B. Comparison with Industry Benchmarks

Key observation: The STAC-T0 record of 13.9 ns does not include any AI component. Our system trades approximately 30 ns of additional latency for the ability to incorporate learned, data-driven trading signals—a trade-off that may be favorable in markets where signal quality outweighs raw speed.

TABLE V
LATENCY COMPARISON

Metric	This Work	STAC-T0	CPU
Tick-to-Trade	$<$50 ns	13.9 ns	3–8 µs
AI Inference	6.2 ns	N/A	5–20 µs
ITCH Parse	7.8 ns	$\sim$15 ns	$\sim$38 µs
Risk Check	1.6 ns	N/A	$\sim$500 ns
Jitter	$\pm$0 cyc	$\pm$1–3 ns	$\pm$0.5–5 µs

C. Neural Network Verification

A Python reference model reproducing the exact weight initialization and INT4/INT8 arithmetic of the RTL implementation is provided. Verification proceeds by comparing the Python model's output logits against the SystemVerilog simulation results for identical input vectors, ensuring bit-accurate correspondence.

VII. LIMITATIONS AND FUTURE WORK

Synthesis validation. The latency and utilization figures reported in this paper are based on architectural analysis and cycle counting. Post-synthesis and post-implementation timing reports from AMD Vivado are required to confirm that the design meets the 644 MHz timing target. A synthesis script is provided but hardware validation has not been performed.

Market data fidelity. The testbenches use synthetic ITCH-like messages. Validation with historical NASDAQ ITCH 5.0 binary data (available from the NASDAQ FTP server) would strengthen the parser's credibility.

Neural network training. The current MLP uses heuristic weight initialization. Integration with a PyTorch training pipeline and deployment via quantization-aware training (QAT) is a natural extension.

Transformer scalability. The linear attention module currently operates on an 8-element sequence with 8-dimensional features. Scaling to longer sequences or larger models requires careful memory management and may exceed single-cycle pipelining constraints.

Multi-symbol support. The current design processes a single symbol. Production systems require concurrent processing of hundreds of symbols, which can be achieved through spatial replication or time-division multiplexing.

VIII. CONCLUSION

We have presented a complete, open-source FPGA-based HFT system that integrates inline neural network inference within the tick-to-trade critical path. The architecture demonstrates that AI-driven trading decisions can be made within a sub-50 ns latency budget, opening the possibility of combining the speed advantage of FPGA execution with the signal quality advantage of machine learning.

By addressing five of the seven identified frontier problems in FPGA trading—inline neural inference, transformer attention, zero-jitter deterministic execution, dynamic session override, and full-stack SmartNIC integration—the system represents a step toward the convergence of hardware acceleration and intelligent trading.

The complete source code, testbenches, and verification infrastructure are available at: https://github.com/Ashutosh0x/fpga-hft-trading-system.

ACKNOWLEDGMENTS

This work was informed by publicly available research from AMD, Exegy, and the open-source FPGA community. The Avellaneda-Stoikov model implementation follows the original 2008 paper. The speculative parser architecture is based on the TechRxiv 2025 publication on parallel ITCH decoding.

REFERENCES

[1] Exegy and AMD, "STAC-T0 Benchmark: 13.9 ns Tick-to-Trade Latency," June 2024. Available: https://www.exegy.com

[2] M. Avellaneda and S. Stoikov, "High-frequency trading in a limit order book," *Quantitative Finance*, vol. 8, no. 3, pp. 217–224, 2008.

[3] R. Pagh and F. F. Rodler, "Cuckoo hashing," *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.

[4] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *Proc. ICML*, 2020.

[5] Y. Umuroglu *et al.*, "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA FPGA*, 2017.

[6] J. Duarte *et al.*, "Fast inference of deep neural networks in FPGAs for particle physics," *Journal of Instrumentation*, vol. 13, no. 07, 2018.

[7] "Speculative parallel ITCH decoding for ultra-low latency FPGA trading systems," TechRxiv Preprint, 2025.

[8] I. Aldridge, *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed. Wiley, 2013.

[9] Exegy, "nxAccess update: 71% execution-stack latency reduction with Session Override," Press Release, April 8, 2026.

[10] AMD, "Alveo UL3524 Accelerator Card for Electronic Trading," Product Brief, 2024.